

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY COURSE
CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS:

Total Marks: 50

Annexure A

Code of Conduct:

*I N.Sai Charan(192525247)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

N.Sai Charan

*Annexure A Scenario-
based*

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

- A. Explain why this issue occurs.
- B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System

A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and Blowfish.

- A. Identify the most suitable algorithm for this scenario.
- B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network

Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

- A. Explain how the key exchange works in this scenario.
- B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

- A. Describe how RSA can be used for encryption and decryption in this scenario.
- B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices

A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

- A. Analyze the constraints of IoT devices.
- B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Cryptography and Network Security (CSA51)

Question 3: Key Exchange in an Unsecured Network

Scenario: Two users want to establish a secure communication channel over a public network using Diffie-Hellman (DH) key exchange, but the network is vulnerable to attacks.

A. How the Key Exchange Works in This Scenario

The Diffie-Hellman protocol allows two parties — say Alice and Bob — to derive a shared secret key over a public, insecure channel without ever transmitting the key itself. The security of the scheme rests on the mathematical difficulty of the discrete logarithm problem. The process unfolds as follows:

1. Public parameter agreement: Alice and Bob first agree on two public values: a large prime number p and a base (primitive root) g , where g is a generator modulo p . These values do not need to be secret and can be sent openly over the network.
2. Private key selection: Each party independently chooses a private, secret integer. Alice picks a secret integer a ; Bob picks a secret integer b . Neither of these values is ever transmitted.
3. Public key computation: Each party computes a public value using modular exponentiation — Alice computes $A = g^a \bmod p$, and Bob computes $B = g^b \bmod p$.
4. Exchange over the public network: Alice sends A to Bob, and Bob sends B to Alice. Since these are transmitted over the compromised network, an attacker can see A and B — but not a or b .
5. Shared secret computation: Alice computes $K = B^a \bmod p = g^{(ab)} \bmod p$; Bob computes $K = A^b \bmod p = g^{(ab)} \bmod p$. Both arrive at the identical shared secret $K = g^{(ab)} \bmod p$, which can be used as a symmetric session key (often after a key-derivation function) to encrypt subsequent communication using an algorithm such as AES.

The strength of this scheme is that even though an eavesdropper on the vulnerable network sees p , g , A , and B , computing a or b from these values (i.e., solving the discrete logarithm problem) is computationally infeasible for sufficiently large primes.

B. Security Threat and Mitigation

Threat: Man-in-the-Middle (MITM) Attack

The critical weakness in plain Diffie-Hellman is that it provides no built-in mechanism for verifying the identity of the party on the other end. Because the network is described as vulnerable, an attacker (Mallory) sitting between Alice and Bob can exploit this lack of authentication:

- Mallory intercepts A (intended for Bob) and instead sends Bob her own public value, A' .
- Mallory intercepts B (intended for Alice) and sends Alice her own public value, B' .
- Alice unknowingly establishes a shared key with Mallory (believing it is Bob), and Bob independently establishes a different shared key with Mallory (believing it is Alice).
- Mallory can now decrypt, read, modify, and re-encrypt every message passing between Alice and Bob, while both believe they are communicating securely and directly with each other.

This is dangerous precisely because the raw DH exchange authenticates nothing — it only guarantees that two parties end up sharing some secret, not that they are sharing it with the party they intended.

Mitigation: Authenticated Diffie-Hellman

To close this gap, the exchange must be bound to verified identities. Practical solutions include:

- Digital signatures: Each party signs their public DH value (A or B) using their private signing key (e.g., RSA or DSA). The other party verifies this signature using a certificate issued by a trusted Certificate Authority (CA) before accepting the value.
- Public Key Infrastructure (PKI) / Certificates: Both parties present X.509 certificates binding their identity to a public key, allowing each to authenticate the other before or during the exchange.
- Station-to-Station (STS) protocol: An enhancement of DH that explicitly incorporates signatures and encryption of the exchanged values to defeat MITM attacks.

- TLS-style deployment: In real-world use (e.g., TLS 1.3), DH or its elliptic-curve variant (ECDHE) is combined with certificate-based authentication, so the exchanged keys are cryptographically tied to verified identities.

Conclusion: Diffie-Hellman alone solves the problem of secret-sharing over an open channel but not the problem of trust. Combining DH with digital signatures and a PKI transforms it into Authenticated DH, which preserves the elegance of the key-agreement scheme while eliminating the MITM vulnerability inherent to the unauthenticated version.

Question 5: Cryptography for IoT Devices

Scenario: A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

A. Analysis of IoT Device Constraints

IoT devices differ fundamentally from traditional computing environments (servers, desktops, even smartphones), and these differences directly shape which cryptographic systems are viable:

- **Limited processing power:** Smart home devices (sensors, thermostats, door locks, cameras) typically use low-cost microcontrollers with limited clock speeds and no dedicated cryptographic co-processors. Computationally expensive operations, like the large modular exponentiations required by RSA, can take a disproportionately long time and drain processing resources needed for the device's primary function.
- **Constrained memory (RAM/ROM/Flash):** IoT devices often have only a few kilobytes to a few megabytes of memory. Algorithms that require large key sizes and large intermediate computation buffers, as RSA does, consume a significant fraction of this scarce memory.
- **Battery/power limitations:** Many smart home devices are battery-powered or designed for extremely low power draw. Every CPU cycle spent on cryptographic computation translates directly into power consumption, so algorithms requiring fewer cycles for equivalent security are strongly preferred.
- **Bandwidth and network overhead:** IoT communication protocols (Zigbee, BLE, LoRaWAN, MQTT over constrained links) often operate over low-bandwidth, high-latency channels. Larger keys and certificates, as required by RSA at strong security levels, mean more bytes must be transmitted, increasing latency and the energy cost of radio transmission.
- **Scale of deployment:** A smart home system may involve many devices needing to establish and manage keys simultaneously, making efficient key generation and exchange important system-wide, not just per device.
- **Physical security limitations:** IoT devices are often physically accessible or removable, increasing the importance of lightweight but strong cryptography that does not force cost-cutting trade-offs elsewhere in the device's design.

Taken together, these constraints mean the ideal cryptographic algorithm for this scenario must deliver strong security per bit of key length, minimize computational cycles, and minimize data transmitted — not simply maximize raw security strength at any cost, as might be acceptable on a server.

B. Recommended Approach: Elliptic Curve Cryptography (ECC)

Recommendation: ECC is the more suitable choice for this smart home IoT scenario, over RSA.

Justification:

- **Smaller key sizes for equivalent security:** ECC achieves comparable cryptographic strength to RSA with dramatically smaller keys — a 256-bit ECC key offers security roughly equivalent to a 3072-bit RSA key. This is a direct consequence of the Elliptic Curve Discrete Logarithm Problem being harder to solve per bit than RSA's integer factorization problem, and it directly addresses the memory constraint identified above.
- **Lower computational overhead:** Because ECC operates on smaller numbers for the same security level, the modular arithmetic operations involved in signing, key exchange, and encryption/decryption require significantly fewer CPU cycles than RSA's large modular exponentiations, translating into faster operations and lower power consumption.

- **Reduced bandwidth consumption:** Smaller keys and signatures mean less data needs to be exchanged over the constrained wireless links typical of smart home networks, reducing radio-on time — usually the most power-hungry operation on an IoT device.
- **Suitability for key exchange:** For establishing session keys between the smart home hub and individual devices, ECDH (Elliptic Curve Diffie-Hellman) provides an efficient, low-overhead key agreement mechanism, often paired with ECDSA for lightweight authentication and signing.
- **Scalability:** Since many IoT devices individually perform key generation and exchange, possibly repeatedly for session rekeying, the cumulative computational and energy savings of ECC over RSA become even more significant across a full smart home deployment with dozens of devices.

Why not RSA: While RSA remains a secure and well-established algorithm, its large key sizes (2048–4096 bits for adequate modern security) impose exactly the kind of memory, processing, and power burden that low-power IoT hardware struggles to sustain. Using RSA here would either force weaker key sizes to stay within device constraints, or work correctly but degrade battery life and responsiveness — both unacceptable trade-offs for a smart home product expected to run unattended for long periods.

Conclusion: Given the smart home system's constraints — limited processing power, memory, and battery life, combined with low-bandwidth wireless communication — ECC (specifically ECDH for key exchange, optionally paired with ECDSA for authentication) is the recommended cryptographic approach. It provides an optimal balance of strong security and resource efficiency, making it the practical choice for real-world IoT deployment, whereas RSA's overhead makes it better suited to environments with abundant computational resources, such as servers or desktop systems.